

CreditTech — Master Project Plan

Alternative Credit Scoring for Financial Inclusion: AI-Driven Lending Models for Farmers, SHGs, FPOs, and Rural Enterprises

One-line project brief: CreditTech is an AI-driven alternative credit-scoring system that turns non-traditional data — SHG repayment history, Account Aggregator bank transactions, satellite-derived crop health, bureau records, and digital activity — into explainable credit-risk scores for financially excluded farmers, SHGs, FPOs, and rural micro-enterprises, validated through a 4–5 village pilot.

This is the single consolidated planning document for the mid-term review — it merges the project-wide conceptual design (problem, architecture, stakeholders, pilot, cost, roadmap) with the execution-level data/ML specification (exact data sources, acquisition steps, feature formulas, training parameters, evaluation and promotion criteria) into one complete reference. Every checklist item on the mid-term submission brief is covered, each in full detail, in one place.

Status convention	Green text = planned/proposed. Text describing code, files, or modules by name refers to work already scaffolded in the team repository. No claim of production-readiness or trained-model results is made anywhere — see §24.
Data legend	REAL/SANDBOX reachable today via a registered API PUBLIC open government/research data SYNTHETIC/PROXY generated or borrowed for pipeline validation only

0. Checklist & Topic Coverage Map

Mid-term checklist item	Covered in
Problem & village-level need assessment	§1.1–1.2
Stakeholder & institutional mapping	§1.3
Existing system (As-Is)	§1.4
Proposed system (To-Be)	§1.5
Complete system architecture	§2
Data architecture & data sources	§3
Technology stack	§14
API/DPI & institutional integration framework	§15
Functional modules	§2.2
User journey / process flow	§10.2
Security, privacy & regulatory framework	§16
Initial 4–5 village pilot design	§19
Preliminary cost & resource requirements	§20
Literature, policy & real-world case evidence	§21

Everything below the checklist line (EDA, preprocessing, feature engineering, labeling, ML training plan with parameters, promotion gate, evaluation, explainability, testing, deployment, reproducibility, roadmap, folder structure) is the execution-level detail the team's own technical planning requires — this is what makes the document replicable end to end, not just presentable.

1. Problem & Objective

2. System Architecture & Functional Modules

3. Data Architecture & Data Sources

4. EDA Plan

5. Data Preprocessing Pipeline

6. Feature Engineering

7. Target-Variable / Labeling Plan

8. ML Model Strategy & Training Plan

9. Model Selection & Promotion Gate

10. Credit Score Design & User Journey

11. Interpretability & Responsible AI

12. Model Evaluation Framework

13. Testing Plan
14. Technology Stack

15. API / DPI & Institutional Integration

16. Security, Privacy & Regulatory Framework

17. Deployment Plan

18. Reproducibility & Versioning

19. Village Pilot Design

20. Cost & Resource Requirements

21. Literature, Policy & Case Evidence

22. Project Roadmap

23. Final Project Folder Structure

24. Exclusions & What Happens Next

25. Assumptions, Risks & Open Questions

26. 2026 State-of-the-Art Research Update

1. Problem & Objective

1.1 The rural credit-access problem

A large share of India's rural households — farmers, Self-Help Group (SHG) members, Farmer Producer Organisation (FPO) members, and micro-enterprises — are **credit-invisible or thin-file**: little or no formal bureau history, irregular/seasonal cash flow tied to crop cycles, and assets (land, livestock) that are hard for a conventional bank process to verify quickly. The result is not "high risk" so much as **unscored** — lenders default to rejection or manual, slow, high-cost field underwriting rather than absorbing the ambiguity.

1.2 Why traditional credit scoring fails here

Traditional scoring assumption	Why it breaks for this population
Stable monthly income	Farm income is seasonal (Kharif/Rabi/Zaid cycles), not monthly-salaried
Formal bureau history (CIBIL etc.)	Most borrowers are first-time formal borrowers — no trade lines to score
Collateral-based lending	Land titles are frequently informal, leased, or community-held; formal registration is slow and often absent
Branch-based, in-person underwriting	Physical field visits are slow (days–weeks) and don't scale to thousands of small-ticket loans
Individual credit history	Ignores strong <i>group-level</i> repayment discipline signals that already exist in SHG/FPO ledgers

1.3 Stakeholder & institutional mapping

Stakeholder	Role	Interaction with CreditTech
Farmer / SHG member / micro-entrepreneur	Borrower	Gives consent, provides data, receives score + reason codes, can appeal
Bank Sakhi / Business Correspondent	Assisted-channel field operator	Submits applications on borrower's behalf via offline-first mobile UI
SHG / FPO federation	Group-level data source	Provides repayment ledgers, meeting attendance, savings records
Partner Regulated Entity (bank / NBFC / MFI)	The actual lender	Consumes the score + reason codes via a handoff API; makes the final lending decision — CreditTech never disburses funds itself
Loan officer (at partner RE)	Internal system user	Reviews score, evidence, and reason codes; approves/rejects/requests more info; every override is logged
Licensed NBFC-Account Aggregator	Regulated data-sharing intermediary	Brokers consented access to bank-transaction data under the RBI AA framework — CreditTech never touches raw bank credentials
Geospatial / bureau data vendors	External data providers	Supply satellite/weather and bureau data via contracted APIs
NABARD	Pilot sponsor / rural-finance apex body	Pilot endorsement; SHG-BLP data reference
RBI / Data Protection Board	Regulator	AA Master Directions, Digital Lending Directions, Fair Practices Code, DPDP Act 2023 govern consent, data use, and grievance handling
Village / Gram Panchayat administration	Local governance	Site coordination for the pilot

1.4 Existing system (As-Is)

Today, underwriting for this population is typically one of: (a) **no formal credit** — informal moneylenders at high interest; (b) **manual branch underwriting** using bureau score where it exists, biased toward rejecting thin-file applicants; or (c) **MFI/SHG-linkage lending** (SHG-BLP) that works but is slow to individualise risk and scale beyond group-liability lending. In all three, decisions take days to weeks, rely on a field visit, and produce no structured, explainable reason a rejected applicant can act on.

1.5 Proposed system (To-Be)

CreditTech is a **decision-support** system (not a lender) that aggregates consented alternative data across five rails — SHG/FPO records, Account Aggregator bank data, satellite/weather (geospatial), bureau data where it exists, and manual field entry — into a "5 Cs" feature set (Character, Capacity, Capital, Collateral, Conditions), produces a calibrated risk score with human-readable, bilingual reason codes, and hands a structured recommendation to a human loan officer at the partner lender. The system never holds credit risk or disburses funds; it shortens and de-biases the underwriting decision, it does not replace it.

1.6 Target users & expected output

Target users: farmers, SHG members, FPO members, and rural micro-enterprise owners (assisted by Bank Sakhis where needed); loan officers at the partner RE; NABARD/RE management (portfolio oversight); the CreditTech engineering/ML team.

Expected output per application: a calibrated probability of repayment, a 0–100 and 300–900 bureau-equivalent score, a Low/Medium/High risk category, the top-5 human-readable reason codes in the local language, and a structured handoff record the partner RE's system can consume — *not* an automated approve/reject decision.

2. System Architecture & Functional Modules

```
Data Sources (AA / Geospatial / Bureau / SHG-FPO manual)
--> Data Ingestion (async orchestrator, per-rail timeout + retry, graceful degradation)
--> Data Storage (PostgreSQL: PII schema-separated from feature tables)
--> Data Processing / EDA --> Feature Engineering --> Feature Tables (versioned)
--> ML Training Pipeline (offline, fixed cadence) --> Model Registry (version + data-snapshot tagged)
--> Prediction / Scoring Service (in-process model load)
--> Credit Scoring Engine (PD -> calibrated score -> risk category)
--> Explainability Layer (SHAP-style reason codes, bilingual)
--> Loan-Officer Dashboard / Institutional Dashboard
--> Monitoring & Feedback (fairness audit, drift, retention worker)
```

2.1 Architecture style

A **modular monolith** (single deployable, domain-separated internal modules) — chosen over microservices because of team size (5–7 engineers) and pilot-scale traffic (<30 requests/day). Microservices would add deployment/observability/networking

overhead with zero benefit at this scale; the monolith preserves a future extraction path (the Data Aggregation module is the natural first seam) without paying that cost now. Single managed PostgreSQL (not NoSQL) because consent/score/decision records need ACID transactions and strong schema for auditability. REST/JSON throughout because every external DPI rail (AA, and eventually OCEN/ULI) is itself REST/JSON. Model inference runs in-process (no dedicated serving cluster/Triton/KServe) — tabular models are small and CPU-fast, and a dedicated serving layer is pure overhead at this inference volume. Single managed cloud, India region, managed containers — not Kubernetes — since the team has no dedicated platform/SRE headcount.

2.2 Functional modules

Module	Responsibility
Consent Service	Create / hash-chain / revoke / verify / audit consent records (DPDP-aligned, append-only cryptographic hash chain — not blockchain, see §16)
Ingestion Orchestrator	Parallel, per-rail-timeout data pulls with graceful degradation (§3.2)
Feature Engineering Pipeline	5-Cs feature computation, WoE encoding, versioned feature tables (§5, §6)
Scoring Service	Feature assembly + scorecard/candidate model inference + calibration (§8, §10)
Explainability Service	SHAP-style contribution → bilingual reason-code rendering (§11)
Handoff Service	Structured export to the partner RE per the agreed API contract (§15)
Monitoring Service	Fairness audit, DPDP data-retention/anonymisation worker (§16)
Simulation Module	Synthetic pilot-cohort generation for demos and load testing (§3.6)

3. Data Architecture & Data Sources

Five acquisition rails feed the feature table. Each degrades independently (§3.2), so a missing rail lowers scoring confidence rather than blocking it. PII (Aadhaar reference, name, phone) is stored schema-separated from the model-ready feature tables from day one — this is the single control that limits breach blast-radius and is enforced at the schema level, not left as a policy note.

Category	What we need	Realistic source & access	Type
Financial / transaction / digital-activity	Bank balance, credit inflows, UPI count/volume, account age, utility timeliness	Account Aggregator — Sahamati/RBI AA ecosystem (Setu, Finvu, CAMSFinServ sandboxes). Register as FIU, obtain sandbox <code>client_id / client_secret</code>	REAL/SANDBOX
Agricultural / farm / weather	NDVI, crop health, rainfall deviation, irrigation access	Primary: contracted geospatial vendor (SLA-backed, contract still open — Phase-0 item E3). Fallback: Sentinel-2 via Copernicus/Google Earth Engine (free, 10 m/~5-day revisit). Rainfall: IMD gridded daily data	REAL / PUBLIC
Bureau	Existing loan status, KCC holding/utilisation, prior bureau score	CIC sandbox (CIBIL/Experian/Equifax/CRIF developer environments); expected low coverage — most borrowers are thin-file by definition	REAL/SANDBOX
Business data	GST filings, business registration	GSTN (GST-registered FPOs/enterprises); Udyam Registration portal (MSME)	PUBLIC
SHG / FPO	Repayment rate, meeting attendance, savings consistency, NABARD grade	Physical passbooks digitised by Bank Sakhi (offline-first entry UI, dual-entry reconciliation); NABARD SHG-BLP published statistics for aggregate cross-check	REAL (field) / PUBLIC
Government / public program	PM-KISAN status, PMFBY enrolment, KCC stats, mandi prices	PM-KISAN & PMFBY portals, data.gov.in (free API key), Agmarknet / e-NAM, NAFIS survey data	PUBLIC
Demographic / geographic	Village/district, agro-climatic zone, landholding band	Census 2011 village-level data, state e-Gram Swaraj/SDG portals	PUBLIC

3.1 Public/research datasets for modeling-technique validation only

These are **not agricultural and not CreditTech's own borrowers** — used solely to validate that the modeling pipeline (encoding, training, evaluation, fairness tooling) works before real pilot outcomes exist. Never presented as CreditTech's own performance.

Dataset	Access	Why used
Kaggle — <i>Home Credit Default Risk</i>	kaggle competitions download -c home-credit-default-risk	Closest public analog: thin-file/alternative-data-style features, real default labels — best pipeline-validation proxy
UCI/Kaggle — <i>German Credit Data</i>	kaggle datasets download -d uciml/german-credit	Small classic scorecard benchmark — cheap unit-testing of WoE/IV and scorecard code path
Kaggle — <i>Give Me Some Credit</i>	kaggle competitions download -c GiveMeSomeCredit	Behavioural/utilisation features analogous to Capacity/Capital feature groups

3.2 Step-by-step acquisition procedure

```

For each application:
1. Borrower gives Aadhaar e-KYC + purpose-scoped consent --> consent hash-chained & stored
2. Orchestrator fires all reachable rails IN PARALLEL (asyncio.gather), each with its own timeout:
   AA connector      : timeout 10s, retry x3, exponential backoff (2s,4s,8s)
   Geospatial conn.  : timeout 15s, no retry (idempotent re-query next cycle if needed)
   Bureau connector   : timeout 8s,  retry x3, exponential backoff
   SHG/FPO           : pre-loaded from last Bank Sakhi sync (no live call at scoring time)
3. Each rail returns DATA, TIMEOUT, or ERROR -- never blocks the others (graceful degradation)
4. Raw payloads validated against a Pydantic schema per rail -- malformed records rejected + logged
5. Validated raw records written to rail-specific staging tables, tagged with a pull_id + timestamp
6. Feature Engineering (Sec 6) consumes staging tables, never raw payloads directly

```

Mirrors the orchestrator already scaffolded in `services/core/ingestion/service.py` and `connectors.py` (parallel fetch, per-rail timeout, tenacity-based retry).

3.3 Geospatial acquisition detail (NDVI)

Register a Google Earth Engine account → define the borrower's farm polygon (or a buffer around the geocoded plot centroid where a polygon isn't available) → filter the `COPERNICUS/S2_SR_HARMONIZED` Sentinel-2 collection by date range + cloud-cover mask → compute NDVI per pixel using bands B8 (NIR) and B4 (RED): $NDVI = (NIR - RED) / (NIR + RED)$ → aggregate (mean) over the polygon → repeat for the prior season for the 2-season trend.

2026 evaluation note (§26.2): we evaluated replacing this direct NDVI computation with a pretrained geospatial foundation model (e.g. Prithvi-EO-2.0) for richer crop-health embeddings. Published cross-region benchmarks (Sec 26 sources) show these models "degrade sharply when tested on a new region" and collapse on minority crops outside their training distribution — not yet reliable for a 5-site, multi-district pilot without local fine-tuning data CreditTech doesn't have pre-pilot. Direct NDVI computation remains primary for the MVP; a foundation-model embedding is a monitored future option, not adopted now.

3.4 What happens when data is scarce or a rail is down

Limitation	Handling strategy
No real repayment labels until post-pilot	V1 model is a deterministic, literature-calibrated scorecard, not a trained classifier (§8.1); real training begins once ≥200 resolved pilot outcomes exist (§7)
Any single rail unreachable	Async orchestrator with independent per-rail timeouts and graceful degradation — a missing rail lowers confidence, it doesn't block scoring
SHG/FPO records are paper-based at many sites	Semi-manual Bank Sakhi entry with dual-entry reconciliation, not assumed API-available at launch
Geospatial vendor contract not yet signed	Open-data fallback (§3.3) validates the connector and NDVI features in parallel with vendor negotiation
Fairness risk in raw data	Caste, religion, health, ethnicity never collected; geography monitored, never a raw model input (§6.2, §16)

3.5 Data at scale

Scale	What changes
10K borrowers	Single Postgres instance + read replica for dashboard queries so reporting never competes with the scoring path
100K	Partition large tables (DataPull, FeatureSnapshot, Score) by time/state; small OLAP store for the dashboard; caching for the geospatial connector
1M+	Logical database separation by domain; sharding by state/region (India's state-level regulatory variation makes this natural anyway)

3.6 Synthetic data (already scaffolded in the repository)

Component	Role
services/core/ingestion/mocks/server.py	In-process mock API server returning synthetic AA/geospatial/bureau payloads — lets the pipeline be built and demoed before real vendor credentials exist
simulate_pilot.py	Synthesises a full borrower cohort to exercise consent → ingestion → scoring → decision end-to-end for demos, CI, and load testing

4. EDA Plan

Designed to run against the feature table (§6) once pilot or proxy data is available. This is a **planned analysis pipeline**, not yet-executed results.

EDA step	What it checks
Data quality checks	Schema conformance, type validation, allowed-range checks per feature (mirrors the Pydantic schemas already used at ingestion)
Missing-value analysis	Missingness rate per feature per rail; distinguish "rail unreachable" (structural) from "field genuinely unknown" (informative)
Outlier detection	IQR/z-score flags on continuous features; domain-sanity bounds (e.g. land holding > 50 acres flagged for review, not auto-dropped)
Duplicate detection	Duplicate applications by Aadhaar-hash + village; duplicate SHG entries from dual-entry reconciliation
Distribution analysis	Histograms/KDE per feature; Kharif vs. Rabi income distribution comparison
Correlation analysis	Feature–feature correlation (multicollinearity check ahead of the logistic baseline); feature–target once labels exist
Class imbalance	Expected default-rate imbalance in the eventual label; resampling/weighting strategy planned in §8.3 rather than assuming balance
Target-variable analysis	Distribution of the default proxy (§7); base rate by segment
Segment-wise analysis	Separate distributions for Farmers / SHG members / FPO members / rural enterprises — structurally different feature availability and income shape
Temporal analysis	Kharif/Rabi/Zaid seasonality in income and NDVI features; month-over-month UPI trend
Geographic analysis	District/agro-climatic-zone aggregates — <i>monitoring only</i> , never a raw scoring feature (§6.2, §16)
Feature–risk relationship	Univariate risk trend per feature — informs WoE binning (§5)

Required visualisations

Missingness heatmap · feature distribution histograms (by segment) · correlation matrix · NDVI/rainfall seasonal trend lines · target base-rate bar chart by segment and district · feature-vs-default-rate bin charts (WoE plots).

Expected insights

Which rails are most often missing at which sites (informs Bank Sakhi prioritisation); which features carry real signal before committing to the candidate model; whether any feature proxies for a protected attribute (checked before modeling, not after).

5. Data Preprocessing Pipeline

```
Raw staging tables --> Schema validation (Pydantic) --> Cleaning (unit/label normalisation)
--> Missing-value treatment --> Outlier treatment --> WoE encoding --> Scaling (baseline model only)
--> Feature construction (derived ratios) --> Dataset validation (assertions) --> Train/Val/Test Split
```

Step	Exact method & why
Validation	Pydantic schemas at ingestion (already in place for API payloads); range/type checks per feature — reject malformed data before it reaches storage

Cleaning	Standardise units (₹, acres), normalise categorical labels (crop names), de-duplicate — inconsistent raw formats break EDA and modeling silently otherwise
Missing-value treatment	Per-feature missingness indicator column preserved (never silently imputed away) + median (continuous) / mode (categorical) imputation <i>within segment</i> — missingness itself can be informative (e.g. no bureau history is expected for a genuinely thin-file applicant); a global mean would blur different income shapes across segments
Outlier treatment	Winsorise continuous features at the 1st/99th percentile within segment; domain-sanity hard bounds flagged for manual review, never auto-dropped — rural income is legitimately skewed and aggressive trimming would bias against real high-income farmers
Categorical encoding (candidate model)	Weight-of-Evidence (WoE): $WoE_{bin} = \ln(\%non\text{-}default_{in\ bin} / \%default_{in\ bin})$, fit on the training fold only, applied unchanged to validation/test — keeps encoding regulator-legible and prevents leakage
Feature selection filter	Information Value (IV): $IV = \sum (\%non\text{-}default - \%default) \times WoE$. Convention: $IV < 0.02$ dropped; 0.02–0.1 weak (kept, monitored); 0.1–0.3 medium; 0.3–0.5 strong; > 0.5 flagged suspicious and manually reviewed for leakage
Scaling	Standardisation (zero mean, unit variance) for the logistic baseline's continuous inputs only; tree-based candidates don't require scaling
Categorical handling — CatBoost path	Native categorical support for <code>crop_type_primary</code> , <code>land_ownership_type</code> , <code>shg_grade</code> , <code>site_type</code> — no encoding needed if CatBoost is selected (§8)
Feature construction	Derived ratios (e.g. <code>kcc_utilization_pct</code> , <code>income_stability_cv</code>) computed once at this stage, versioned alongside raw inputs
Dataset validation	Post-pipeline schema + range re-check; row-count and null-rate assertions — becomes the <code>tests/test_features/</code> suite (§13)
Train/val/test split	Stratified by target label AND grouped by <code>village_id</code> (StratifiedGroupKFold) — no village's borrowers appear in both train and validation, preventing village-level leakage; nominal 70/15/15; time-aware — validation/test rows come from a resolution date after training rows', respecting the crop-cycle sequence

6. Feature Engineering

The catalogue is organised under a "5 Cs" framework (40+ features drafted in `docs/phase0/preliminary_feature_list.md`). Below: category summary, representative calculation logic, and predictive rationale.

Category	Representative features	How calculated	Why it predicts credit risk
Character	<code>shg_repayment_rate</code> (C001), <code>upi_transaction_regularity</code> (C008)	$\text{on_time_repayments} / \text{total_scheduled}$ (trailing 12mo); CV of monthly UPI txn count	Behavioural discipline is a strong repayment-intent signal even without bureau history
Capacity	<code>estimated_crop_income_kharif/rabi</code> (CA001/2), <code>income_stability_cv</code> (CA003)	NDVI-derived yield proxy $\times$ cultivated area $\times$ mandi price (Agmarknet); $\text{std}(\text{income}) / \text{mean}(\text{income})$	Cash-flow stability matters more than absolute income for seasonal borrowers
Capital	<code>bank_balance_avg_6m/min_6m</code> (CP002/3), <code>kcc_utilization_pct</code> (CP006)	Mean/min end-of-day balance, trailing 6mo; $\text{outstanding}/\text{limit} \times 100$	Savings buffer indicates shock-absorption capacity
Collateral	<code>land_holding_acres</code> (CL001), <code>land_quality_ndvi_avg</code> (CL004)	Manual/DigiLocker entry; mean NDVI over farm polygon, 2 seasons	Even informal land signals recoverable value and cultivation seriousness
Conditions	<code>ndvi_trend_2season</code> (CO001), <code>rainfall_deviation_pct</code> (CO002)	$\text{NDVI}(t) - \text{NDVI}(t-1)$; $(\text{actual} - \text{block LPA}) / \text{block LPA} \times 100$ (IMD)	External agro-climatic shocks are the dominant risk driver for agri-linked income
Digital activity	<code>upi_monthly_txn_count/volume</code> (D001/2)	Mean monthly count/sum, trailing 6mo, from AA transaction ledger	Digital footprint proxies for formal-economy integration and stability

6.1 Reference/baseline values for SHAP-style contributions

Each feature has a cohort reference value (rural-borrower-cohort mean) against which an applicant's contribution is measured — e.g. `shg_repayment_rate` reference = 90.0, `bank_balance_avg_6m` reference = ₹5,000. Already defined in `ml/evaluation/explain.py`; must be recalibrated once real pilot-cohort statistics exist.

6.2 Fairness, privacy, leakage & regulatory flags

Concern	How it's handled
Protected-attribute leakage	Caste, religion, health, ethnicity are never collected as features, by design (not merely excluded post-hoc)
Geography as a proxy	Village/district/agro-climatic zone computed and <i>monitored</i> for demographic parity, never a raw scoring input
Gender / landholding	Monitored dimensions in the fairness audit, not raw model features driving the score for demographic-parity purposes
Temporal leakage	Post-loan repayment behaviour must never enter pre-loan feature computation — enforced by the time-aware split (§5) and a dedicated leakage test (§13)
PII exposure	PII schema-separated from the feature table from day one; structured logs auto-scrubbed of PII
Consent scope creep	Every ingestion pull is checked against an active, purpose-scoped consent record before it runs

7. Target-Variable / Labeling Plan

Decision	Specification
Primary default proxy	DPD90+ — 90 or more days past due on a scheduled repayment, measured on the partner RE's loan-management ledger
SHG-routed loan proxy	Where individual DPD tracking doesn't apply cleanly, use missed repayment across a full crop cycle (Kharif or Rabi) as the default event
Observation window	Outcome labelled only once the loan has had at least one full scheduled-repayment cycle; loans within their first cycle are <i>censored</i> , not "good," and excluded from training until resolved
Minimum data volume to begin training	≥ 200 resolved outcomes after ≥ 6 months of pilot operation — below this, sampling noise makes any trained model unreliable; the deterministic scorecard remains the only production model
Known proxy risk (documented, not hidden)	DPD90+ can misclassify hardship-driven late payment (e.g. a failed monsoon) as equivalent to unwillingness to repay — a real fairness/regulatory-alignment risk in an agricultural portfolio, reviewed alongside the fairness gate (§9)

8. Machine Learning Model Strategy & Training Plan

Model	Role	Verdict
Logistic Regression (weighted "5 Cs" scorecard)	Baseline / V1 production fallback	Interpretable by construction; deployed first since no training labels exist pre-pilot (deterministic, literature-calibrated weights, not yet trained)
Decision Tree	Diagnostic only	Quick univariate risk-splits during EDA; too unstable alone for production
Random Forest	Candidate benchmark	Good sanity-check ensemble; generally outperformed by boosting on tabular data at this scale
XGBoost / LightGBM	Primary candidate (post-pilot)	Best expected accuracy/AUC on tabular alternative data; already listed as an optional dependency for this purpose
CatBoost	Optional alternative	Strong native categorical handling (crop type, SHG grade); evaluate if XGBoost/LightGBM underperform on categoricals
Neural networks	Not justified for V1	Tabular data at pilot scale (hundreds–low thousands of rows) doesn't benefit from deep learning; adds opacity the regulatory context doesn't reward
Tabular foundation models (TabPFN v2 / TabICL v2) — 2026 addition, §8.5	Secondary evaluation candidate, small-N window	In-context learners requiring no hyperparameter tuning; published benchmarks on 14 PD datasets show strongest advantage specifically under ~2,000 observations — a close match to early-pilot data volume; run alongside XGBoost, never auto-promoted

8.1 Baseline — exact specification (already scaffolded)

Not a trained model pre-pilot — a deterministic, hand-set weighted logistic function, already implemented in `ml/training/scorecard.py`:

```
log_odds = intercept (-1.2) + Σ (weight_i × feature_value_i)
prob_repayment = 1 / (1 + exp(-log_odds))
score_100 = round(prob_repayment * 100, 1)
score_900 = offset + factor * ln( prob_repayment / (1 - prob_repayment) )
    where factor = 20 / ln(2), offset = 600 - factor * ln(50)  # PD0=20, 50:1 odds at 600
```

Recalibration once real outcomes exist: refit intercept + weights via maximum-likelihood logistic regression (`sklearn.linear_model.LogisticRegression`, `penalty='l2'`, `C` tuned by grid `[0.01, 0.1, 1, 10]` via 5-fold CV on

AUC) on the WoE-encoded feature set, keeping the same feature list for continuity of the reason-code templates.

8.2 Candidate models — hyperparameter search space (post-pilot trigger, §7)

Model	Hyperparameter search space	Notes
XGBoost	<code>max_depth</code> : [3,4,5,6,8] · <code>learning_rate</code> : [0.01,0.05,0.1] · <code>n_estimators</code> : up to 500, <code>early_stopping_rounds</code> =50 · <code>subsample</code> : [0.7,0.8,0.9,1.0] · <code>colsample_bytree</code> : [0.6,0.8,1.0] · <code>min_child_weight</code> : [1,3,5,10] · <code>reg_alpha</code> : [0,0.1,1] · <code>reg_lambda</code> : [1,5,10] · <code>scale_pos_weight</code> = neg/pos ratio	Primary candidate; <code>objective='binary:logistic'</code> , <code>eval_metric=['auc','aucpr']</code>
LightGBM	<code>num_leaves</code> : [15,31,63] · <code>learning_rate</code> : [0.01,0.05,0.1] · <code>feature_fraction</code> : [0.6,0.8,1.0] · <code>bagging_fraction</code> : [0.7,0.9] · <code>min_child_samples</code> : [10,20,50] · <code>lambda_l1/l2</code> : [0,1,5] · <code>is_unbalance</code> =True	Alternative candidate — faster on larger pilot-scale datasets
CatBoost	<code>depth</code> : [4,6,8] · <code>learning_rate</code> : [0.01,0.05,0.1] · <code>l2_leaf_reg</code> : [1,3,5,9] · <code>iterations</code> : up to 500 with early stopping · native <code>cat_features</code> supplied	Evaluate only if XGBoost/LightGBM underperform on categorical-heavy segments

8.3 Search strategy, imbalance handling & reproducibility

Setting	Value / method
Hyperparameter search	Bayesian search via Optuna, 100 trials per candidate model, optimising validation PR-AUC (primary, due to class imbalance), with AUC and Expected Calibration Error tracked as secondary constraints
Cross-validation	5-fold StratifiedGroupKFold (grouped by <code>village_id</code> , stratified by target) — same scheme as the train/val/test split (§5)
Class-imbalance handling	Primary: <code>scale_pos_weight</code> / <code>is_unbalance</code> / <code>class_weights</code> — cost-sensitive learning, not resampling. SMOTE-style oversampling is explicitly not default (leakage risk on grouped/temporal folds)
Early stopping	50 rounds without validation-metric improvement, on the held-out fold within each CV split
Random seed policy	<code>random_state</code> =42 fixed across every split, model, and search process — pinned in the shared training config
Post-hoc calibration	Isotonic regression (preferred with enough resolved outcomes) or Platt scaling (fallback for small bins), evaluated via reliability diagrams + Brier score (§12)
Compute	CPU-only (tabular gradient-boosted trees at this row count don't benefit from GPU); library versions pinned via <code>pyproject.toml</code> 's <code>ml_extra</code>

8.4 Problem framing

Task	Used?	Why
Binary classification (default / no default)	Yes	Core modeling task once labels exist
Probability of Default (PD)	Yes	Calibrated probability output, needed for the score (§10)
Credit-risk score / risk categories	Yes	PD transformed into 300–900 score and Low/Medium/High category
Regression	No (V1)	Loan-amount/limit regression is a plausible V2 extension, out of MVP scope
Clustering / segmentation	Diagnostic only	Used in EDA to understand sub-populations, not a production model
Anomaly detection	Diagnostic only	Flags suspicious/inconsistent applications for manual review

8.5 2026 addition — tabular foundation models as a bounded secondary candidate

Research published in 2026 on foundation models for credit-risk prediction (benchmarking TabICL and TabPFN v2 against gradient-boosted trees across 14 probability-of-default and 7 loss-given-default datasets) found these in-context tabular learners “generally perform best” without any hyperparameter tuning, with the advantage concentrated specifically in the **<2,000-observation regime** — exactly where CreditTech sits for the first 1–2 years of pilot data. Beyond ~8,000 observations the advantage disappears and gradient-boosted trees (CatBoost/XGBoost) are competitive or better.

Decision	Rationale
Adopt as a parallel candidate, not a replacement	Once ≥50 resolved outcomes exist (below the \$7 200-case threshold for the main candidate-training trigger), run TabICL v2 alongside the baseline as a low-cost secondary read — it needs no training loop, only in-context inference
Use TabICL v2, not RealTabPFN	RealTabPFN's weights carry a non-commercial license, incompatible with a system feeding real lending decisions; TabICL v2 is fully open-source
Explainability gap — unresolved	The source paper explicitly flags that no established SHAP-equivalent exists yet for these architectures, while Basel/GDPR/EU-AI-Act-style expectations require explainable risk models — a TFM output is therefore diagnostic-only and cannot enter the promotion gate (\$9) until an explainability method is validated for it
Not a promotion path on its own	The same \$9 gate (AUC gain, KS, calibration, fairness, human sign-off) applies to any model considered for production — a TFM's zero-shot convenience does not exempt it from that gate

9. Model Selection & Promotion Gate

A candidate model is promoted to replace the baseline as production scorer **only if every condition below holds** — beating baseline accuracy alone is explicitly insufficient.

Gate condition	Threshold
Discrimination improvement	Candidate AUC ≥ Baseline AUC + 0.03, statistically significant (DeLong test, $p < 0.05$)
Rank-ordering strength	KS statistic ≥ 0.30
Calibration quality	Brier score ≤ Baseline Brier score; Expected Calibration Error ≤ 0.05
Fairness — demographic parity	Approval-rate gap across gender / landholding-band / geography ≤ 5 percentage points
Fairness — equalised odds	True-positive-rate gap across the same dimensions ≤ 7 percentage points
Minimum evaluable sample	≥ 200 resolved outcomes overall, ≥ 30 per segment separately evaluated — below this the segment is reported "insufficient data," never silently passed
Human sign-off	Promotion is human-approved after the automated gate passes — never a fully automatic deploy

If a candidate fails any condition, the deterministic baseline remains in production and the candidate's evaluation report is logged for the next retraining cycle — a failed candidate is diagnostic information, not a dead end.

10. Credit Score Design & User Journey

```
Applicant Data --> Alternative Data (AA/Geo/Bureau/SHG) --> Feature Engineering
--> ML Model (baseline scorecard, or candidate post-pilot) --> Probability of Default (PD)
--> Score Calibration --> Risk Score (0-100, 300-900) --> Risk Category (Low/Med/High)
--> Reason Codes --> Lending Recommendation to Loan Officer
```

PD is transformed via log-odds calibration (Points-to-Double-Odds = 20, base odds 50:1 at score 600) into a 0–100 score and a familiar 300–900 bureau-equivalent scale, then bucketed into Low/Medium/High risk categories with defined thresholds. **This is a decision-support score, not an approval** — handed to a human loan officer with evidence (\$11.1).

10.2 User journey / process flow

```
1. Borrower approaches Bank Sakhi or opens app -> 2. Aadhaar e-KYC + AA consent capture
3. Consent recorded (hash-chained, revocable) -> 4. Parallel data pull (AA / Geospatial / Bureau / SHG)
5. Graceful degradation if a rail is unreachable -> 6. Feature computation + scoring
7. Score + confidence band + bilingual reason codes -> 8. Loan officer reviews on decision dashboard
9. Officer approves / rejects / requests more info (override reason logged if any)
10. Structured handoff sent to partner RE -> 11. Borrower notified; appeal channel available
```

Not claimed: that this score is regulator-approved or production-ready. It is a pilot-stage decision-support output, subject to the fairness gate, human review, and external regulatory sign-offs tracked separately (\$21.1).

11. Interpretability & Responsible AI

Mechanism	Approach
Baseline (linear) explanations	Exact Shapley decomposition: $SHAP_i = coefficient_i \times (value_i - reference_i)$ — already implemented in <code>ml/evaluation/explain.py</code> ; exact because the model is additive/linear

Candidate (tree) explanations	<code>shap.TreeExplainer</code> from the open-source <code>shap</code> library, required once the model is non-linear. Background sample: 200 randomly sampled training rows
Global explanations	Mean absolute SHAP value per feature across the validation set, published alongside each model version in the registry
Reason codes	Top-5 features by absolute SHAP value per application, mapped to a bilingual (English/Hindi) template library (<code>docs/phase0/shap_reason_code_templates.md</code>)
Bias / fairness analysis	Weekly batch audit of demographic parity + loan-officer override rate across gender, landholding band, geography
Sensitive-feature handling	Caste/religion/health/ethnicity never collected; geography monitored, never a raw feature (§6.2)
Data privacy	PII/feature schema separation; DPDP-aligned storage-limitation worker that anonymises PII once consent lapses (§16)
Consent	Append-only, cryptographically hash-chained consent ledger — create/verify/revoke/audit, purpose-scoped per rail
Human-in-the-loop review	Every score reaches a loan officer with evidence and reason codes before any lending decision; overrides logged, not silently accepted

11.1 How a loan officer understands a score

The officer's dashboard shows the score, risk category, and top-5 reason codes in plain local-language sentences (e.g. "SHG repayment rate is 92% — positive factor" / "NDVI trend declining over last 2 seasons — caution factor"), each traceable to a specific feature and its contribution — not a black-box number.

12. Model Evaluation Framework

Why accuracy alone is insufficient: the default class is expected to be rare (imbalanced), so a model predicting "no default" for everyone can score high on raw accuracy while being useless — and harmful, since it hides exactly the applicants who need a nuanced decision. Evaluation combines ranking quality, calibration, and fairness, not a single number.

Metric	Computed how	Why it's included
ROC-AUC / PR-AUC	<code>sklearn.metrics.roc_auc_score</code> / <code>average_precision_score</code> on held-out fold	PR-AUC specifically compensates for class imbalance
KS statistic	Max separation between cumulative good/bad distributions across score deciles	Standard credit-scorecard rank-ordering metric, familiar to lending stakeholders
Precision / Recall / F1	At the selected operating threshold (§12.1)	Names the false-positive/false-negative trade-off explicitly
Calibration curve + Brier score	10-bin reliability diagram; <code>sklearn.metrics.brier_score_loss</code>	The score is presented as a probability-derived number — it must mean what it says
Confusion matrix	At selected threshold, per segment	Makes FP/FN cost trade-off visible per segment
Segment-wise re-computation	Every metric above, recomputed per borrower segment and pilot site	An aggregate-only metric can hide poor performance on one segment
Fairness metrics	Demographic parity gap, equalised-odds gap (§9)	Mandatory pre-promotion gate, not a post-hoc report
Cross-validation	Stratified k-fold by segment; time-aware folds by season	Respects the crop-cycle structure of the data

12.1 Threshold selection

The operating threshold (score cut-off separating Low/Medium/High risk) is chosen against the partner RE's stated cost ratio of a missed default vs. a wrongly-denied applicant — not fixed at 0.5 — using a cost-weighted threshold sweep over the validation fold.

13. Testing Plan

Test type	Covers
Unit tests	Individual functions — scorecard math, SHAP-style contribution calc, consent hash-chain verification
Data-validation tests	Schema/range checks reject malformed or out-of-bound inputs
Feature-engineering tests	Derived features compute correctly on known inputs (WoE bin edges, ratio calculations)
Model tests	Scorecard produces a probability in [0,1]; calibration output stays in [300,900]; candidate model reproducible from a fixed seed
API tests	Consent, ingestion, scoring, handoff, monitoring endpoints — status codes, auth, payload shape

Integration tests	Ingestion orchestrator against mocked AA/geospatial/bureau servers, including partial-rail-failure paths
End-to-end tests	Full consent → ingest → score → officer decision → handoff flow on a synthetic borrower
Edge cases	Missing/incorrect applicant data, all-rails-down scoring, duplicate applications, expired consent mid-pull
Model robustness	Score stability under small, realistic input perturbations
Fairness tests	Automated check that demographic-parity gap stays within tolerance before any model is promoted
Data-leakage tests	Assert no post-loan field can influence a pre-loan feature computation
Security / privacy tests	PII never appears in logs; access-control on PII endpoints; consent required before every ingestion pull

Five categories are already scaffolded as pytest suites (`tests/test_consent.py`, `test_ingestion.py`, `test_scoring.py`, `test_monitoring.py`, `test_handoff.py`, `test_simulation.py`) — this plan extends that structure.

```
tests/
  test_data/           # schema, range, duplicate, missingness checks
  test_features/       # feature-engineering + WoE binning correctness
  test_models/         # scorecard math, calibration bounds, candidate reproducibility
  test_api/            # consent / ingestion / scoring / handoff endpoint tests
  test_pipeline/       # preprocessing + feature-construction pipeline, end-to-end
  test_integration/    # orchestrator against mocked rails, partial-failure paths
  test_fairness/       # demographic-parity + leakage checks
```

14. Technology Stack

Layer	Technology	Status
API framework	Python, FastAPI	MUST-HAVE
Data / feature tooling	Pandas, NumPy	MUST-HAVE
Baseline modeling	Scikit-learn (logistic regression, encoders)	MUST-HAVE
Candidate modeling (post-pilot)	XGBoost / LightGBM; CatBoost optional	LATER
Explainability	SHAP (open-source library)	MUST-HAVE (post-pilot candidate)
Notebooks	Jupyter (EDA, feature-engineering exploration)	MUST-HAVE
Database	PostgreSQL + SQLAlchemy (async) + Alembic migrations	MUST-HAVE
Containerisation	Docker, Docker Compose (local dev with mocked connectors)	MUST-HAVE
Version control / CI	Git/GitHub, GitHub Actions (lint, type-check, test, build)	MUST-HAVE
Infra-as-code	Terraform, single-region managed containers (no Kubernetes)	MUST-HAVE
Testing	pytest, pytest-asyncio, ruff, mypy	MUST-HAVE
Data/model versioning	DVC (once real training data exists)	LATER
Experiment tracking — 2026 update	MLflow (open-source, lightweight, standard for small-team ML lifecycle tracking)	MUST-HAVE (post-pilot candidate)
Feature store — 2026 addition	Feast (open-source) — central, versioned serving point for the 5-Cs feature table once it's consumed by more than one model/service	LATER — add once candidate training starts
Drift & fairness monitoring tool — 2026 addition	Evidently (open-source) — concretises the "weekly batch fairness/drift job" (§17) into a maintained, dashboarded tool rather than bespoke scripts	MUST-HAVE
Data validation tool — 2026 addition	Deepchecks (open-source) — automates the dataset-validation assertions in §5, beyond ad-hoc Pydantic checks	OPTIONAL — Pydantic covers MVP; add if validation logic grows
Tabular foundation model runtime — 2026 addition	TabICL v2 (open-source, MIT-style license) — see §8.5	LATER — bounded secondary candidate only
Dashboard UI	FastAPI + Jinja2/HTMX (internal officer tool, 20–50 users)	MUST-HAVE
Borrower app	Android, Kotlin or React Native (offline data capture)	MUST-HAVE
Streamlit/React for analytics	—	OPTIONAL
Model serving infra (Triton/KServe)	—	NOT NEEDED at this traffic volume

Rows marked "2026 update/addition" come from a review of current open-source MLOps tooling (§26.4) — chosen because each is free, lightweight, and matched to a small-team pilot rather than an enterprise platform (e.g. Databricks Feature Store, Kubeflow) that would be over-engineering at this scale.

15. API / DPI & Institutional Integration

```
External Data / API (AA, Geospatial vendor, Bureau/CIC, SHG federation)
--> API Layer (per-connector client: auth, timeout, exponential-backoff retry)
--> Validation (schema + consent-scope check)
--> Feature Pipeline --> ML Model --> Credit Score --> Lending Recommendation
--> Structured Handoff to Partner RE (REST/JSON, per agreed contract spec)
```

REST/JSON is used throughout because every relevant external Digital Public Infrastructure rail (Account Aggregator, and eventually OCEN/ULI-style handoff) is itself REST/JSON — matching it avoids an unnecessary translation layer.

15.1 Current status of India's digital lending rails (2026 update)

Framework	Purpose	Status as of this review
Account Aggregator (AA)	Consent-based sharing of banking/GST/income data across institutions	Well-established — multiple FIPs/FIUs already integrated under RBI's regulatory framework. This is why AA is CreditTech's primary financial-data rail, not a future one
OCEN (Open Credit Enablement Network)	Standardised APIs connecting lenders, agents, and data providers for cash-flow-based unsecured lending	Live pilots running (GeM Sahay, GST Sahay, Jan Aushadhi Kendra, and private-network pilots) — nearer-term integration target than ULI for the eventual RE handoff contract (§21.1)
ULI (Unified Lending Interface)	Standardised access to borrower financial and non-financial data (banks, government databases, land records) for underwriting	Still in development , with only limited pilot implementations for specific data sources — not yet live. CreditTech's REST/JSON handoff design is ULI-compatible in shape, but ULI integration work is deliberately deferred until pilot adoption data justifies it, not built speculatively against an interface still in flux

Explicitly flagged: actual API availability, authorisation, consent scope, and regulatory sign-off for each rail (AA FIU licensing, geospatial vendor contract, bureau CIC access, RE handoff contract) must be verified and formally agreed before any production integration — tracked as open external items (§21.1), not assumed.

16. Security, Privacy & Regulatory Framework

16.1 Applicable regulatory frameworks

RBI Digital Lending Directions (LSP/RE model, default-loss-guarantee cap at 5%); DPDP Act 2023 + Rules 2025 (consent, breach notification, retention); Account Aggregator Master Directions (FIU role, "data-blind" AA design); Credit Information Companies Act 2005 (bureau data). **Not applicable:** HIPAA (no health data), PCI-DSS (CreditTech never touches payment card/fund flows), children's-data rules (borrowers are adults). (Engineering framing, not legal advice — validated with counsel per Phase-0 item E5.)

16.2 Engineering implications

- Consent capture must be free, specific, informed, unconditional, and revocable — the revocation path is built *before* launch, not as a fast-follow.
- Data-breach reporting within regulatory timelines requires an incident-response runbook live before go-live, not improvised during an incident.
- Data minimisation is enforced at the schema level (explicit feature-exclusion list, §6.2) — a design decision, not a policy document.
- Consent records retained ≥7 years — storage/retention lifecycle planned into the schema from day one.

16.3 Threat model

Threat	Mitigation	Priority
OTP/e-KYC brute force	Rate limiting, attempt lockout, Aadhaar-mandated biometric fallback	High
PII leakage via logs/backups	Structured logging with PII scrubbing, encrypted backups, separate PII/feature stores	Critical
Compromised AA/OCEN API credentials	mTLS, credential rotation, least-privilege service accounts	High
Model-input manipulation (score gaming)	Input validation, anomaly detection on feature distributions, human-in-loop decisioning	Medium
Proxy-variable bias creeping into features	Explicit feature-exclusion list, fairness gate before every promotion, independent audit	Critical

Insecure SHG/FPO manual-ingestion pathway	Field-level validation, dual-entry/spot-check reconciliation, audit log on manual edits	Medium
Dependency/supply-chain vulnerabilities	Automated dependency scanning in CI (pip-audit, gitleaks), pinned versions	Medium
SSRF via geospatial/vendor connector	Allow-list outbound domains, connector runs in a restricted network segment	Medium
Bank Sakhi device compromise (shared/public devices)	Short session TTLs, explicit logout flows, no persisted credentials on assisted devices	Medium

Consent ledger design: append-only PostgreSQL table with cryptographic hash-chaining per record — not a permissioned blockchain. A hash-chained table gives tamper-evidence and auditability without DLT's operational complexity, consensus overhead, or unfamiliar tooling for a small team; CreditTech is the system of record for a single institution's pilot, not a multi-party consortium requiring decentralised trust.

17. Deployment Plan (Realistic MVP)

Stage	Approach
Development environment	Docker Compose locally, all four rails mocked, live-reload FastAPI + Postgres
Model training	Offline, fixed cadence (not continuous) — pilot scale doesn't justify streaming retraining
Model serialisation	Versioned artifact + training-data snapshot reference in a lightweight model registry
Backend API	FastAPI monolith, containerised
Database	Single managed PostgreSQL instance, India region
Frontend / dashboard	Server-rendered officer dashboard (Jinja2/HTMX); Android borrower app
Containerisation	Docker images built and scanned in CI (pip-audit, gitleaks)
Cloud deployment	Single managed cloud, India region, managed containers (ECS/Cloud Run) — not Kubernetes
Monitoring	Weekly batch fairness/drift job; structured, PII-scrubbed logs; managed APM (bought, not built)
Model / data drift	Season-boundary and significant-drift triggers for retraining review (human-approved before promotion)
Retraining strategy	At most one or two retrains across a 9–12 month pilot window (one Kharif–Rabi cycle) — fully automated retraining is explicitly premature

MVP-essential: Docker Compose dev environment, CI pipeline, single-region managed deploy, weekly batch monitoring. **Deferred:** autoscaling, multi-region, real-time streaming monitoring, dedicated model-serving infrastructure.

18. Reproducibility & Versioning

Element	Policy
Random seeds	Single fixed <code>random_state=42</code> , set once in a shared config, not per-script
Library versions	Pinned in <code>pyproject.toml</code> 's <code>ml</code> extra — exact versions locked for training code
Data versioning	DVC (or equivalent) pointed at <code>ml/registry/</code> once real training data exists — every run references an immutable data-snapshot hash
Model registry entry	Model artifact + hyperparameters + data-snapshot reference + full evaluation report, versioned and never overwritten
Config-driven training	All hyperparameter grids and thresholds live in a single versioned YAML config, not hard-coded in scripts

```
model: xgboost
seed: 42
cv:
  scheme: stratified_group_kfold
  n_splits: 5
  group_col: village_id
search:
  method: optuna
  n_trials: 100
  objective_metric: pr_auc
  space:
    max_depth: [3,4,5,6,8]
    learning_rate: [0.01,0.05,0.1]
    n_estimators: {max: 500, early_stopping_rounds: 50}
    subsample: [0.7,0.8,0.9,1.0]
    colsample_bytree: [0.6,0.8,1.0]
promotion_gate:
```

```
min_auc_gain: 0.03
min_ks: 0.30
max_ece: 0.05
max_fairness_gap_pp: 5
min_resolved_outcomes: 200
```

19. Initial 4–5 Village Pilot Design

Element	Plan
Participants	~750–1,250 borrowers across 4–5 sites: farmers, SHG members, FPO members, rural micro-enterprises; onboarded via Bank Sakhi-assisted channel and/or Android app
Data collected	All five rails where reachable at each site — a per-site connectivity/device assessment precedes onboarding
Consent	Aadhaar e-KYC + purpose-scoped, revocable consent recorded in the hash-chained ledger before any data pull
System output	Score + confidence band + bilingual reason codes, delivered to the single partner RE integrated for the pilot
User interaction	Bank Sakhi submits/assists onboarding; loan officer reviews score + evidence on the decision dashboard and records approve/reject/request-more-info with mandatory override-reason logging
Pilot success metrics	Approval-rate uplift vs. the RE's existing process; score-availability rate under partial-rail data; PAR90 on disbursed loans; reason-code comprehension (borrower survey); fairness-parity gap within tolerance
Feedback loop	Resolved repayment outcomes from site 1 feed the post-pilot candidate-model training trigger (\$7, \$9); grievance/appeal outcomes reviewed alongside officer override patterns

Site 1 launches first and must be stable for 4+ weeks (including a production-credential smoke test on every external rail) before expansion to sites 2–5 — a staged rollout, not a simultaneous 5-site launch.

20. Preliminary Cost & Resource Requirements

Order-of-magnitude planning estimates for a 9–12 month, 4–5 site pilot — **not** a committed budget. Excludes credit capital, disbursed by the partner RE, never by CreditTech.

Cost driver	Notes
Core engineering team	Largest single line item at pilot stage (backend/ML engineers, one mobile developer, field-ops coordination)
Geospatial (EO) data licensing	Per-hectare/per-query vendor pricing, scales with borrower count; open-data fallback (\$3.3) reduces this during pilot validation
Account Aggregator per-consent fees	Scales directly with applications processed
Cloud infrastructure & MLOps	Grows step-wise across scale stages — protected by avoiding Kubernetes/dedicated model-serving infra at pilot scale
Field operations	Bank Sakhi/BC coordination and device procurement at each site — real, ongoing, not eliminated by technology, only made more efficient

20.1 Team & roles

Role	Responsibility
Backend / API engineer(s)	Consent, ingestion, scoring, handoff services
ML engineer	Feature engineering, baseline scorecard, candidate model (post-pilot), evaluation/fairness tooling
Mobile developer	Android borrower app (offline-first)
Field-ops coordinator	Bank Sakhi training, device/connectivity assessment, site liaison
Project/product lead	Stakeholder coordination, regulatory-item tracking (\$21.1), pilot sequencing

21. Literature, Policy & Real-World Case Evidence

21.1 Policy & regulatory grounding

- **RBI Account Aggregator (AA) Master Directions** — governs consented financial-data sharing via licensed NBFC-AAs; basis for the AA connector and consent-scope design.
- **Digital Personal Data Protection (DPDP) Act, 2023** — basis for the hash-chained consent ledger, PII/feature schema separation, and the storage-limitation worker.
- **RBI Fair Practices Code** — basis for the mandatory grievance/appeal channel and SLA.

- **NABARD SHG-Bank Linkage Programme (SHG-BLP)** data and grading — source of the SHG-grade feature and a real-world precedent for group-liability alternative underwriting at scale.

21.2 Related academic work

The "5 Cs" feature-weighting approach draws on published alternative-credit-scoring literature (Jonnalagadda & Babu, 2026) for the initial bootstrap weights, used in the absence of real repayment labels. A 2026 systematic literature review on ML-powered credit scoring (Springer *Artificial Intelligence Review*) and current research on tabular foundation models for credit-risk prediction inform the modernisation choices in §26.

21.3 Real-world case evidence — comparative review of related open-source projects

Project	Relevant technique	Adopted into this plan?
agri-access (Indonesia)	Open-weight satellite foundation model (Prithvi-EO-2.0) against free Sentinel imagery instead of a paid vendor	Yes — informs the open-data geospatial fallback (§3.3)
Krishi-Patra (NABARD hackathon)	Dynamic, ongoing crop-health monitoring tied to loan terms; blockchain for transparency	Monitoring concept adopted as a post-pilot enhancement; blockchain not adopted — the hash-chained ledger already gives tamper-evidence at lower operational cost (§16.3)
Credit-Risk-Probability-Model-for-Alternative-Data	WoE/IV encoding, explicit proxy-label documentation, Basel-II framing, DVC data versioning	Yes — informs §5/§8 encoding strategy and §12 evaluation discipline

22. Project Roadmap

Phase	Focus	Key tasks	Deliverables	Tools
1	Problem & literature review	Stakeholder mapping, policy review, related-project benchmarking	This document	Desk research
2	Data collection	Set up AA/geospatial/bureau sandboxes; assemble public datasets; build mock server for synthetic data	Raw data pull scripts, mock API server	httpx, Requests, public portals
3	EDA	Run §4 EDA plan on proxy/synthetic data	EDA notebook + insights report	Jupyter, Pandas, Matplotlib/Seaborn
4	Data processing	Implement §5 pipeline	Cleaned, validated dataset	Pandas, Pydantic
5	Feature engineering	Build 5-Cs feature table + WoE encoding	Versioned feature table	Pandas, scikit-learn
6	Baseline models	Deploy logistic scorecard	Working scorecard + calibration	scikit-learn
7	Advanced models	Train XGBoost/LightGBM on labelled/proxy data	Candidate model + comparison report	XGBoost, LightGBM
8	Explainability & fairness	Wire SHAP; run fairness audit	Reason-code templates, fairness report	SHAP, custom fairness auditor
9	Evaluation	Run §12 metric suite + cross-validation	Evaluation report	scikit-learn metrics
10	API & deployment	Wire FastAPI services, Docker Compose, CI/CD	Deployable service	FastAPI, Docker, GitHub Actions
11	Testing	Implement §13 test suites	Passing CI pipeline	pytest, ruff, mypy
12	Pilot / validation	Site-1 staged launch, production-credential smoke test	Pilot results, feedback loop	Monitoring dashboards

23. Final Project Folder Structure

```

credittech/
  data/           # raw + processed datasets (synthetic/proxy for pilot demo)
  notebooks/     # EDA, feature-engineering exploration (Jupyter)
  src/           # shared application code
  features/      # feature-engineering pipeline (5-Cs table, WoE encoding)
  models/        # baseline scorecard + candidate model training code
  api/           # FastAPI services: consent, ingestion, scoring, handoff, monitoring
  frontend/      # officer dashboard (Jinja2/HTMX), Android borrower app
  tests/         # see Sec 13 test structure
  configs/       # environment configs, training config YAML (Sec 18)
  docs/         # phase0 specs, roadmap, this document, benchmark research

```

```
deployment/ # Dockerfile, docker-compose.yml, Terraform (infra/)
README.md   # project overview + local setup
```

Mirrors the structure already scaffolded in the team repository (`services/core/*`, `ml/*`, `apps/*`, `infra/*`, `docs/*`) — this plan builds on existing scaffolding rather than proposing a rewrite.

24. What This Document Excludes, and What Happens Next

No trained-model results, accuracy numbers, or score distributions appear in this document — none exist yet. The V1 production model is the deterministic scorecard (§8.1), not a trained classifier; candidate-model training (§8.2) has an explicit data-volume trigger (§7) that has not yet been reached. Presenting invented numbers here would misrepresent the project's actual state.

What happens	Trigger condition
Baseline scorecard goes live	Pilot launch (site 1), immediately — no training data required (§8.1)
Baseline recalibration (refit weights on real data)	First batch of resolved outcomes, even before the 200-case candidate threshold
Candidate model training begins	≥ 200 resolved outcomes AND ≥ 6 months of pilot operation (§7)
Candidate evaluation report produced	Immediately after training — presented regardless of whether the promotion gate (§9) passes
Results/metrics presented	Once the above exist — the natural "final result" section for a later submission, deliberately not fabricated here

25. Assumptions, Risks & Open Questions

Item	Note
AA/geospatial/bureau sandbox access	Assumed obtainable during the build phase even where production vendor contracts (Phase-0 external items) are still pending — sandbox tiers are typically free/low-friction
Sentinel-2 fallback resolution	10 m/~5-day revisit is coarser than a commercial vendor's SLA — acceptable for pilot validation, not asserted as production-equivalent
Proxy-label risk (DPD90+)	Documented explicitly in §7 — a known, not hidden, source of potential misclassification in an agricultural portfolio
Public benchmark datasets	Used only to validate pipeline mechanics — never conflated with CreditTech's own population or presented as pilot results
Sample size at pilot scale	Hundreds, not thousands, of resolved outcomes in year one — exactly why the interpretable baseline remains the production fallback even after a candidate is trained (§8.1, §9)
Single-vendor geospatial dependency	Mitigated by the open-data fallback (§3.3) but not eliminated — tracked as Phase-0 external item E3
Single partner RE for the pilot	Sufficient for MVP validation; multi-RE routing explicitly deferred to post-pilot
Open external regulatory/business items	Signed RE Letter of Intent, AA FIU credentials, geospatial vendor contract, NABARD endorsement, legal sign-off on the consent flow, Bank Sakhi device procurement — dependencies outside the team's direct control, tracked separately, not assumed resolved

Source artifacts merged into this document: `Credittechplanningcontext.md` (full engineering roadmap) · `docs/phase0/*` (regulatory specs) · `docs/phase0/preliminary_feature_list.md` (complete 40+ feature table) · `docs/research/external_benchmark_analysis.md` (comparative review) · `CreditTech_MidTerm_Conceptual_Design_Plan.pdf` and `CreditTech_Data_ML_Training_Specification.pdf` (the two prior documents this master plan consolidates).

26. 2026 State-of-the-Art Research Update

Before finalising this plan, the team reviewed current (2026) research and tooling across five areas touching CreditTech, to adopt genuine improvements and — just as importantly — to document what was evaluated and deliberately **not** adopted, and why. This section is the record of that review; the inline updates it produced are cross-referenced from §3.3, §8.5, §14, and §15.1.

26.1 Tabular foundation models for the small-N pilot problem — ADOPTED (bounded)

2025–26 saw tabular foundation models (TabPFN v2, TabICL v2) move from proof-of-concept to genuine contenders: in-context learners needing no training loop or hyperparameter search. A 2026 benchmark against gradient-boosted trees across 14 probability-of-default and 7 loss-given-default datasets found these models win most folds without tuning, with the advantage concentrated specifically under ~2,000 observations — a near-exact match to CreditTech's own bottleneck (§7: real training only begins at ≥200 resolved outcomes, and stays small for the first 1–2 pilot years). **Decision:** adopt TabICL v2 (open license, unlike RealTabPFN's non-commercial terms) as a bounded secondary candidate once ≥50 outcomes exist, run diagnostically alongside the baseline — not eligible for production promotion until an explainability method for it is validated,

since no established SHAP-equivalent exists yet for these architectures and Basel/GDPR/EU-AI-Act-style expectations require explainable risk models. Full detail in §8.5.

26.2 Geospatial foundation models for crop health — EVALUATED, NOT ADOPTED

Models like Prithvi-EO-2.0, SpectralGPT, and SatMAE were considered as a richer alternative to direct NDVI computation. A 2026 cross-region benchmark found all three "degrade sharply when tested on a new region" — one model's crop-segmentation accuracy fell from ~26% to ~7% moving from North Carolina to California, and minority crops (e.g. winter wheat) scored near-zero across nearly all models tested, without local fine-tuning data. **Decision:** keep direct Sentinel-2 NDVI computation (§3.3) as primary for the MVP — it is simple, auditable, and doesn't inherit an unvalidated model's regional failure modes across a 4–5-district pilot with genuinely different agro-climatic zones. Revisit if an India- or crop-specific fine-tuned variant becomes available and is independently validated on pilot-adjacent regions.

26.3 India's digital lending rails — AA mature, OCEN piloting, ULI still forming

Confirmed via current sources (§15.1): Account Aggregator is well-established with FIPs/FIUs already integrated — validating it as CreditTech's primary financial-data rail today, not a future bet. OCEN has several live pilots (GeM Sahay, GST Sahay, Jan Aushadhi Kendra) and is the more realistic near-term target for the RE handoff contract than ULI, which remains in development with only limited pilot data-source coverage. **Decision:** no architecture change — REST/JSON was already the right choice (§2.1) — but the roadmap's ULI-adoption trigger is now grounded in a specific, current ecosystem state rather than a general assumption.

26.4 MLOps tooling — modernised from generic names to a concrete, small-team stack

The previous technology-stack draft listed MLflow as merely "optional" and had no named feature-store or drift-monitoring tool. Current (2026) guidance for small teams building a first ML pipeline converges on a specific lightweight open-source stack, adopted in §14: **Feast** (feature store), **MLflow** (experiment tracking — upgraded from optional to must-have once candidate training starts), **Evidently** (drift/fairness monitoring — gives the existing "weekly batch fairness job" a real, dashboarded tool instead of bespoke scripts), and **Deepchecks** (data validation). All four were chosen specifically because they are free and lightweight, avoiding enterprise platforms (Databricks Feature Store, Kubeflow) that would be over-engineering at pilot scale — consistent with the project's existing build-vs-buy discipline (§2.1).

26.5 Explainable & fair AI governance — validated existing design, sharpened language

Current literature on trustworthy credit scoring for emerging markets converges on patterns CreditTech's design already implements: SHAP/LIME-style local+global explanations, continuous model governance (versioned retraining/validation — §18), multi-layered bias auditing (§9, §16), and alternative-data diversification for thin-file borrowers (§3). No architecture change follows from this review — it confirms the existing fairness-gate and consent design were already aligned with current practice rather than needing to catch up to it.

26.6 Sources

Foundation Models for Credit Risk Prediction: arxiv.org/html/2605.18147v1 · State of Tabular Foundation Models (2026): mindfulmodeler.substack.com · Benchmarking Geospatial Foundation Models for Agriculture: arxiv.org/html/2606.29664 · ULI/OCEN/AA framework comparison: pn.ispirit.in · AI-Powered Credit Scoring for Emerging Markets: tmasolutions.com · ML-powered credit scoring systematic review, *AI Review* (Springer, 2026): link.springer.com · Open-source MLOps tools 2026: cake.ai